MSML610: Advanced Machine Learning

Lesson 09.4: gh Filter

Instructor: Dr. GP Saggese, gsaggese@umd.edu

References:

- AIMA 14: Probabilistic reasoning over time
- https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python

- ***gh Filter***

Tracking Objects

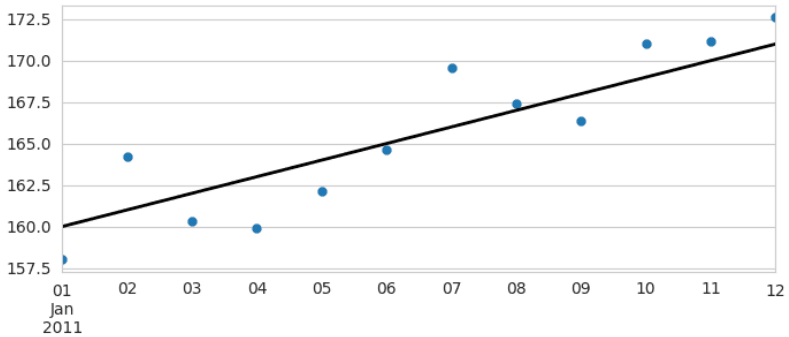
- Many problems can be formulated as **tracking objects**
- **Examples**
  - Navigation of aircraft, drones, autonomous cars
  - Robotics: arm kinematics to predict the position of joints
  - Sensor fusion: merge multiple sensor readings
  - Finance: predict economic variables
  - Computer vision: track moving objects across video
- **Kalman filter**
  - Used for state estimation in dynamic systems with noisy, uncertain measurements
  - Track state over time using model to make predictions and observations

Some Guiding Principles

- **The world is noisy**
  - E.g., a car might swerve around a pothole or brake for a pedestrian
  - E.g., wind or ice might change the car's path
- **Sensors are noisy**
  - E.g., a kitchen scale gives different readings for the same object
- **Knowledge is uncertain**
  - E.g., you alter beliefs on how things work based on evidence and its strength
- **Combine past information and system knowledge** to estimate future information
  - E.g., if a car moves at speed X at time t, the speed at time $t+1$ is $\approx X$
- **Data (even if noisy) $>$ guess**
  - Never discard information, no matter how poor it is
  - E.g., two sensors, even if one is less accurate, are better than one

Example of Estimating Body Weight

- **Goal**: estimate your weight over time while going to the gym
- Two possible approaches
  1. **Predict your weight**
     - Track calorie intake
     - Track energy expense
     - Compute expected weight gain
     - *Cons*: Difficult to track food intake and exercise accurately
  2. **Measure your weight**
     - Use a scale
     - *Cons*: Scale is noisy, water weight fluctuates, different clothes
- **Problem**: Prediction doesn't match measurements
  - E.g., at time t:
    - Estimate $\hat{x}_{t|t-1} = 159$ based on calorie intake and energy expended
    - Scale measures 164



Example of Estimating Body Weight

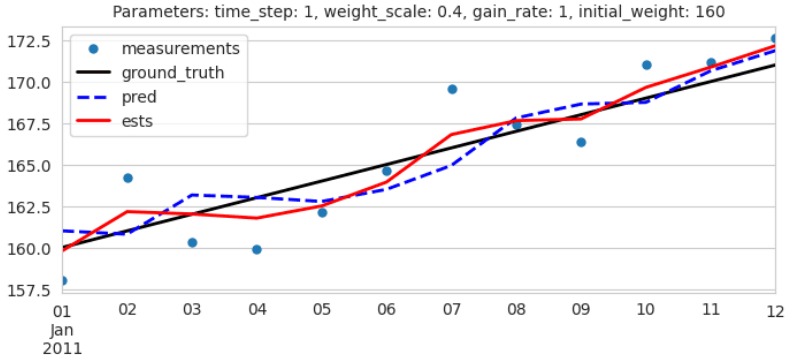
- **What's the real weight?**
  - You need to blend prediction and measurement

$$\text{estimate} = 0.6 \times \text{prediction} + (1 - 0.6) \times \text{measurement}$$

  - Because of coefficient 0.6 you believe the prediction is more likely correct than the measurement
  - The prediction is always between the prediction and the measurement
- **Algorithm**
  1. Start with an initial guess
     - Assume it's correct for now
  2. Predict the next weight based on the model
  3. Measure the weight
  4. Estimate the next weight by merging values:
  5. Go back to first step

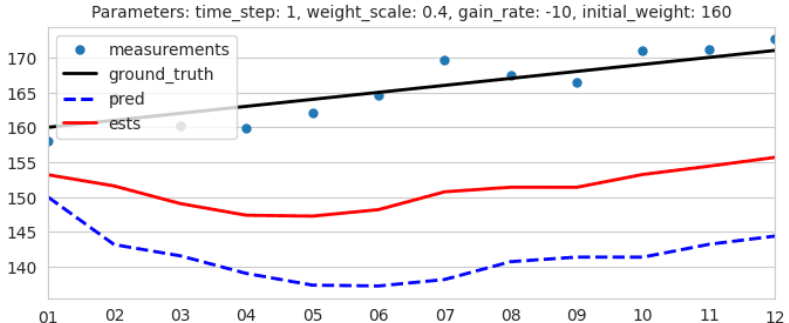
Example of Estimating Body Weight

- Assume you know the initial weight and the gain rate
  - Black line is the actual weight (i.e., *ground truth*)
  - Red line is the *prediction* from previous day's weight
  - Green circles are the *measurements*
  - Blue line is the *estimate* from the filter
    - Always falls between measurement and prediction

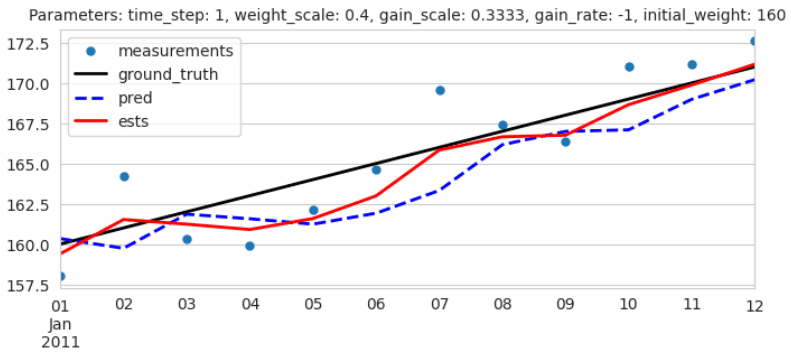


- Not impressive since the prediction model describes the ground truth

- When the model predicts a gain of -10lb/day (incorrect)
  - Estimates diverge from measurements



Example of Estimating Body Weight



- Two **problems**
  - The filter needs a correct guess of the weight change rate
  - The rate of change can vary over time
- **Solution**: estimate the rate of change from measurements
  - "Data is better than a guess, even if it's noisy"
  - Refine the estimate of the gain rate:

    new gain = old gain + 0.3 (measurement - prediction) / 1 day

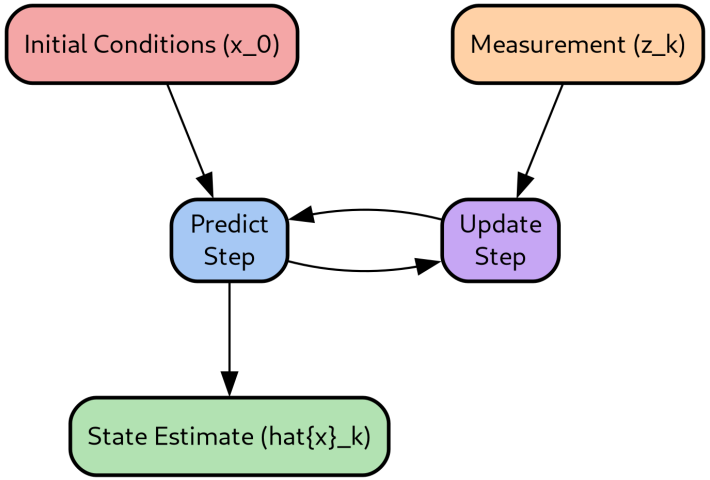
- The "state" is given by `weight` and `gain_rate`
  - You need to predict and update both

g-h Filter

- The previous algorithm is called **g-h filter**
  - g: scaling used to blend predicted state and measurement
  - h: scaling used to update the parameter of the system model based on the measurements
- g-h filters have different values of g and h to achieve different properties
  - E.g., pick g to minimize the transient error when the derivative of the signal has a step (i.e., a discontinuity of the slope)
- Many filters (including Kalman filter) are just generalizations of a g-h filter

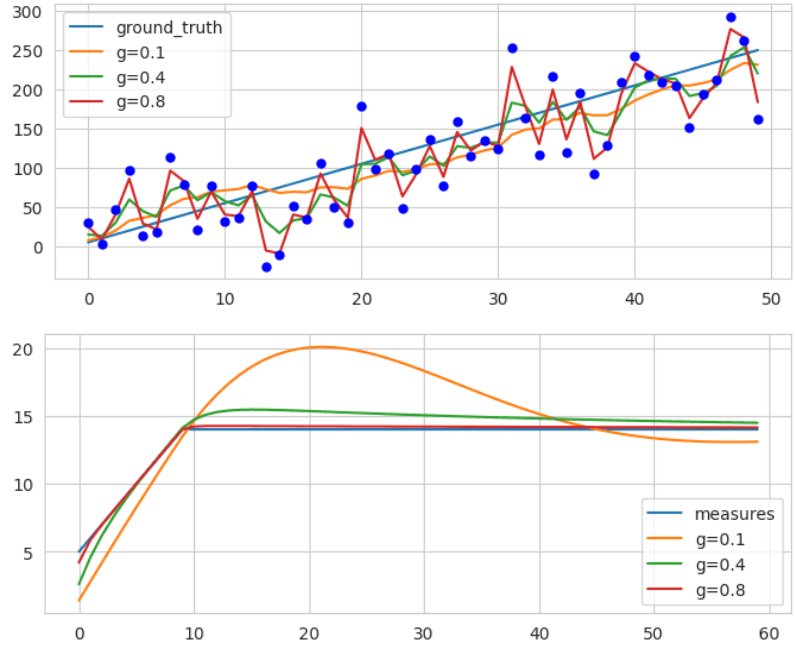
g-h Filter Algorithm: Pseudo-Code

1. **Initialization**
   - Initialize state of the filter
   - Initialize belief in the state
2. **Predict**
   - Use system model to predict state at next time step
   - Adjust belief to account for uncertainty in prediction
3. **Update**
   - Get measurement and associated belief about its accuracy
   - Use as estimate of the next state a point between estimated state and measurement



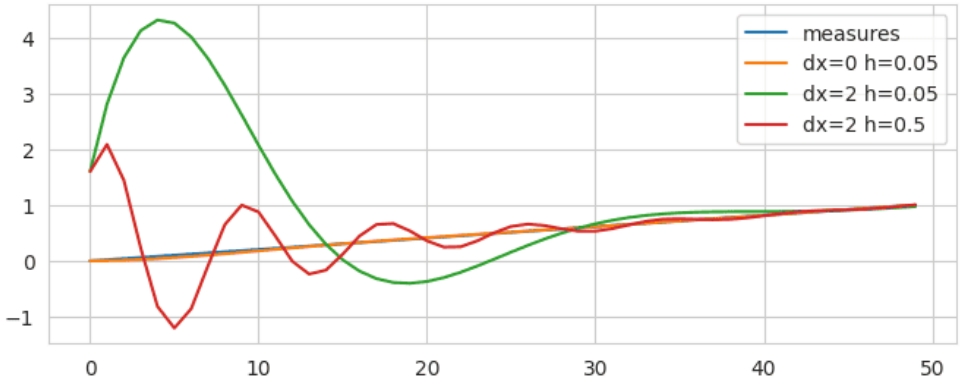
Interpretation of g

- If $g = 0$:
  - Follow system model, ignore measurements
- If g is small:
  - Follow model more than measurements
- If g increases:
  - Follow measurements more, ignore prediction
  - Useful with accurate measurements, inaccurate model
- If $g = 1$:
  - Follow only measurements, ignore model
  - Too large $\rightarrow$ follow measurements, reject no noise



Interpretation of h

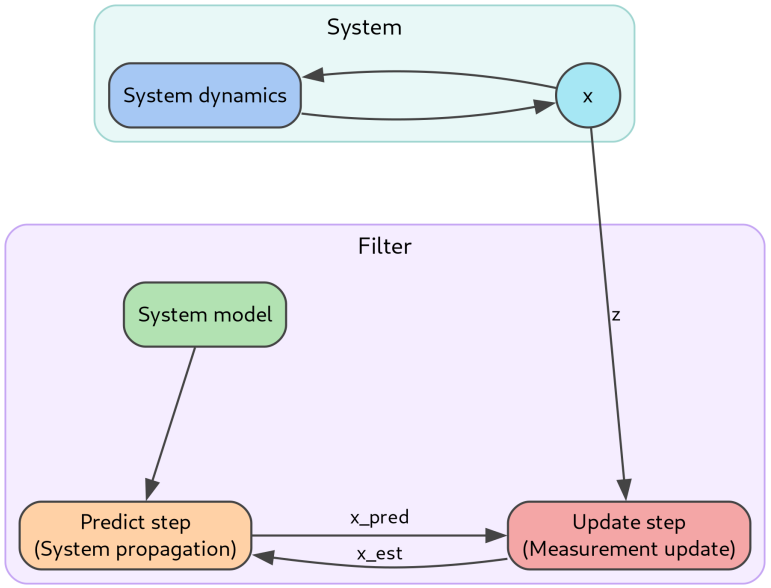
- Estimate model parameters from data:
  - Change of weight
  - Rate of change of measurements
  - Car speed on different terrains



- If $h = 0$:
  - Filter follows previous rate of change
  - Adapts slowly to signal changes
- If $h = 1$:
  - Filter reacts rapidly to significant signal variations
- If h is:
  - Smaller $\rightarrow$ Smoother, slower rate adaptation
  - Larger $\rightarrow$ Faster rate adjustment
- **Note**: Incorrect initial state is similar to a changing state

Control Theory Nomenclature

- State space models developed in control theory → different nomenclature
- **System**: object to estimate/track
- **Filter**: algorithm to estimate system state
- **State of the system** `x`: current values of interest
  - Part of the state might be hidden



- **Measurement** `z`: measured value of system
  - Observable, can be inaccurate
- **State estimate** `x_est`: filter estimate of state
- **System model**: mathematical model of system
  - Typically has error
- **System propagation**: predict step using system model to form new state estimate `x_pred`